

IMPLEMENTATION BRIEF

Registered Device + WebAuthn + GPS Attendance

VE WorkLog - Vertex Electronics and Vision Engineering

Required result

An employee can mark normal attendance only when all three checks pass: (1) the employee is logged in, (2) an admin-approved WebAuthn credential completes device user verification, and (3) GPS confirms the device is within the office radius. A password alone must not be enough.

1. Existing application context

Item	Current value
Application	Existing Node.js/Express VE WorkLog portal with PostgreSQL and JWT login
Production origin	https://tasks.vertex.pk
Office coordinates	31.441300523433583, 74.32441912480384
Office radius	Approximately 100 metres; validate on the server
Attendance status rules	Before 7:00 AM blocked; after 9:35 AM Late; at/after 11:00 AM Half Day
Monthly rules	3 lates = 1 absence; 2 half-days = 1 absence
Testing state	Employee Attendance navigation is currently hidden; admin Attendance remains available

2. Security objective

The current GPS check proves that a device is at the office but does not prove who is operating it. WebAuthn adds possession of an approved credential and device-level user verification.

Attendance decision

Approved credential + WebAuthn user verification + valid office GPS = accept attendance. If any normal verification step fails, do not silently accept attendance; offer the existing manual request/admin approval process.

3. Employee device registration

1. Employee signs into VE WorkLog on the phone intended for attendance.
2. Employee opens Attendance and selects Register this device.
3. Server generates a short-lived, single-use WebAuthn registration challenge.
4. Browser invokes the platform authenticator and requests fingerprint, Face ID, device PIN, pattern, or another supported verification method.
5. The private key remains on the device. The public credential response is sent to the server.
6. Server verifies the challenge, expected origin, relying-party ID, and user-verification result.
7. Credential is saved as Pending and appears in the admin Device Approvals screen.
8. Admin approves or rejects it. Policy: one active approved attendance credential per employee.

4. Check-in and checkout

1. Employee selects Check In or Check Out.
2. Server creates a fresh authentication challenge bound to the employee and requested action.
3. Browser asks the approved authenticator for user verification and returns a signed assertion.
4. Browser obtains latitude, longitude, and GPS accuracy.
5. Frontend sends the signed assertion and GPS payload to the attendance endpoint.
6. Server verifies WebAuthn first, calculates office distance, checks GPS accuracy, and then applies attendance time/status rules.
7. Server records attendance plus credential ID, verification time, coordinates, accuracy, IP, and user agent for audit.
8. Challenge is consumed so it cannot be replayed.

5. Employee and admin screens

Screen/state	Required behavior
No device	Show Register this device; block normal check-in/out
Pending approval	Show device and Awaiting admin approval; manual request remains available
Approved	Show normal attendance controls; require WebAuthn for every check-in and checkout
Rejected/revoked	Block normal attendance and offer new/replacement registration
Admin Device Approvals	List employee, device label, browser, registration time, status, Approve/Reject/Revoke
Manage Users	Show device status alongside the existing attendance-enabled toggle

6. Proposed database additions

Table	Essential fields
attendance_credentials	id, user_id, credential_id, public_key, counter, transports, device_name, browser_info, status, registered_at, approved_by, approved_at, revoked_at, revocation_reason
webauthn_challenges	user_id, challenge, purpose/action, expires_at, consumed_at
attendance audit additions	credential reference, verification mode, GPS accuracy, IP address, user agent, verification timestamp

7. Required API endpoints

Endpoint	Purpose
POST /api/webauthn/register/options	Create registration challenge/options
POST /api/webauthn/register/verify	Verify and store pending credential
POST /api/webauthn/auth/options	Create check-in/checkout authentication challenge
GET /api/webauthn/device	Return current employee device status
POST /api/webauthn/device/replace	Request replacement with mandatory reason
GET /api/webauthn/admin/devices	Admin device list
PUT /api/webauthn/admin/devices/:id/approve	Approve pending credential
PUT /api/webauthn/admin/devices/:id/reject	Reject pending credential
PUT /api/webauthn/admin/devices/:id/revoke	Revoke approved credential
POST /api/attendance/checkin	Verify WebAuthn + GPS and record check-in
POST /api/attendance/checkout	Verify WebAuthn + GPS and record checkout

8. WebAuthn production settings

Setting	Value
---------	-------

Setting	Value
rpName	VE WorkLog
rpID	tasks.vertex.pk
expectedOrigin	https://tasks.vertex.pk
userVerification	required for registration and authentication
authenticatorAttachment	platform preferred
residentKey	preferred
attestationType	none initially
Suggested Node library	@simplewebauthn/server; follow the installed-version API
Challenge policy	Single-use, action-bound, employee-bound, recommended expiry 3-5 minutes

9. Critical limitations

- A website cannot reliably force fingerprint-only or Face-ID-only verification. With userVerification required, the device may allow fingerprint, face, device PIN, pattern, computer password, or security-key PIN.
- VE WorkLog receives only a cryptographic verification result; it does not receive or store biometric data.
- If an employee gives the approved phone and its PIN to a colleague, WebAuthn may still succeed.
- Some passkeys synchronize through cloud credential managers, so a passkey is not always proof of one exact physical phone.
- For stronger protection against deliberate cooperation, add a live selfie/liveness check or use dedicated office biometric hardware.

10. Manual fallback and replacement

- If WebAuthn or GPS is unavailable, retain the existing mandatory-reason manual request and admin approval flow.
- Manual records must be labelled unverified by WebAuthn.
- Employee device replacement requires a mandatory reason and admin approval.
- Admin must be able to revoke a lost device immediately; the old credential must stop working after revocation.
- Preserve the existing rule allowing checkout while manual check-in approval is still pending.

11. Files expected to change

File	Change
package.json	Add @simplewebauthn/server; rebuild the Docker image because dependencies change
db/schema.sql	Add credential/challenge tables and attendance audit columns
server/routes/webauthn.js	New registration, authentication, device, and admin routes
server/routes/attendance.js	Require verified assertion for normal attendance and retain manual fallback
server/index.js	Mount /api/webauthn
public/app.js	Device registration/status, WebAuthn ceremony, admin approval UI
public/style.css	Styles for device states and approvals

12. Acceptance tests

- Correct account password on an unapproved phone cannot mark normal attendance.
- Approved credential with failed user verification cannot mark attendance.
- Approved credential outside the office radius cannot mark normal attendance.
- Expired, reused, wrong-user, or wrong-action challenge is rejected.
- Rejected/revoked credential is rejected.
- Android Chrome, iPhone Safari, and Windows Hello are tested where supported.
- Existing Present/Late/Half Day/absence calculations and daily/monthly/PDF reports still work.
- Manual attendance and Early Leave / Remote remarks continue through admin approval.

13. Rollout

1. Implement device registration and admin approval first without enforcing it.
2. Pilot with a small group across Android and iPhone, plus Windows if desktop attendance is supported.
3. Enable WebAuthn enforcement for pilot employees while retaining manual fallback.
4. Restore Attendance in both employee navigation definitions (empViews and empltems) when testing resumes.
5. After stable operation, enforce it for all attendance-enabled employees.

Estimated production-ready implementation: approximately 3-5 working days, including cross-device testing. A basic MVP may be available in about 2 working days but should be piloted before becoming official attendance.

Non-negotiable server rule

Do not trust a client-provided biometric-success flag, office-distance result, or attendance status. Generate and verify challenges on the server, verify origin/RP ID/credential ownership/status, calculate GPS distance on the server, and consume every challenge after use.

14. Current deployment note

The application runs in Docker on the Oracle Ubuntu VM (vv_app, vv_db, vv_nginx). Adding a Node package requires rebuilding/recreating the app image; copying JavaScript files into the running container is not enough for new node_modules. The production environment file is /home/ubuntu/vertex-monitoring-app/.env and must remain outside source control.